Empirical Analysis of Transformers on Graph Connectivity

Part IV: What Makes a Graph Hard, and a Similarity Read-out

Edoardo Paccagnella

June 13, 2026

Abstract

We ask what makes a graph hard for a standard connectivity transformer, and whether a non-standard read-out helps. Difficulty has two separable axes — distance-vs-capacity ($3^L = 9$) and density-vs-training, not the spectral gap — and a similarity read-out $\hat{R}_{ij} \propto \cos(h_i, h_j)$ doubles the reach radius to $2 \cdot 3^L$ and removes the model's over-connection error, but not a bottleneck.

1 Introduction

We rely on two findings from the earlier reports: the *data lever* does not help a standard transformer generalise (Report III: the outcome is seed-dominated), and an L-layer model hits a sharp wall at shortest-path distance $\approx 3^L$ ($= 9$ for our $L = 2$ models) — a property of the model, not the data. We ask three things, in order: (1) is the diameter the right notion of difficulty (Section 3); (2) can the spectral gap be isolated as a separate cause (Section 4); (3) does a *spectral* read-out move the wall (Section 5). Family-level numbers are averaged over several seeds (four at $n = 40$, eight at $n = 20$); only the $n = 64$ depth sweep is single-seed.

2 Setup

Model. The RoBERTa-faithful standard transformer of Report III ($L = 2$, single head, $d_{\text{model}} = 512$, normalised-ReLU attention), trained online at $n = 20$ for 10^6 steps. Two training sets: **ER** (ER$(20, 0.08)$, as in the paper) and **mixed** (a uniform mixture of the families below); two read-outs (*linear*, *similarity*); an optional Laplacian-smoothness loss.

Two quantities. The **diameter** is the largest shortest-path distance (how far the farthest pair is). The **spectral gap** is the smallest non-zero eigenvalue of the normalised Laplacian: small means a bottleneck (a thin bridge or long corridor), large means a well-mixed graph.

2.1 The graph families

A panel chosen to spread across diameter and gap (Figure 1). All graphs are generated in `data.py` on n nodes without self-loops (the self-loops and a random node permutation are added at evaluation time). How each is built:

- **ER**: each off-diagonal edge present independently with probability p (here $p = 0.08$) — the upper triangle is a Bernoulli(p) mask, then symmetrised. This is training set A.
- **1chain**: the single path $0{-}1{-}\cdots{-}(n{-}1)$ — one component, diameter $n{-}1$, tiny gap.
- **1cycle**: 1chain plus the closing edge $(0, n{-}1)$ — the cycle C_n, every node degree 2, diameter $\lfloor n/2 \rfloor$.

- **2chains**: $n = 2k$ nodes split into two halves $\{0, \ldots, k-1\}$ and $\{k, \ldots, 2k-1\}$, each made into a path — two components.
- **2cliques**: the same two halves, each made into a complete graph.
- **2cycle**: the same two halves, each made into a cycle C_k.
- **path_union**: the n nodes are cut at $k-1$ random points into $k \in \{1, \ldots, 4\}$ contiguous segments, each turned into a path. With $k = 1$ this is a single path of all n nodes (distances up to $n-1$); $k > 1$ adds genuine cross-component disconnections, so "predict all connected" is wrong.
- **er_blocks / clique_blocks**: the same random partition into $k \in \{1, \ldots, 4\}$ blocks; each block is made a complete graph (clique_blocks) or a *connected* ER graph — a random spanning path over the block (to guarantee connectivity) plus extra edges at probability 0.3 (er_blocks).
- **barbell**: two cliques of size $c \approx n/3$ placed at the two ends, joined by a single path running through all the middle nodes — one component, but every path between the two cliques must cross that one bridge (the smallest gap of all). *barbell_var* sweeps the gap by varying c at fixed shape (small $c \Rightarrow$ long bridge $\Rightarrow$ tiny gap; large $c \Rightarrow$ short bridge $\Rightarrow$ larger gap).
- **expander**: an approximate random d-regular graph ($d = 3$) — start from a random Hamiltonian cycle (a random node order joined consecutively, which guarantees connectivity and degree 2), then add extra random near-perfect matchings up to degree d. Tiny diameter, large gap — the structural opposite of a path. *expander_var* varies d.
- **chain_plus**: a long path of $n-s$ nodes plus a separate short path of s nodes, with $s \sim U[3, n/3]$ — two components, with within-component distances up to $n-s-1$ (well past 9), so it exposes the 3^L wall even at $n = 20$.

The mixed training set is an *online* stream (graphs generated on the fly, no fixed set): a family is drawn uniformly per sample from the nine families *er, er_blocks, clique_blocks, path_union, 2chains, 2cliques, 1cycle, 2cycle, 1chain*. *expander*, *barbell* and *chain_plus* are held out, so they probe unseen structure. (With batch 1000 and 10^6 steps the model sees $\sim 10^9$ graphs in total; note that the deterministic families — 1chain, 1cycle, 2chains, 2cliques, 2cycle — are a single graph each up to a node permutation, so the model sees the same structure re-permuted, while ER, the blocks and path_union are genuinely varied.)

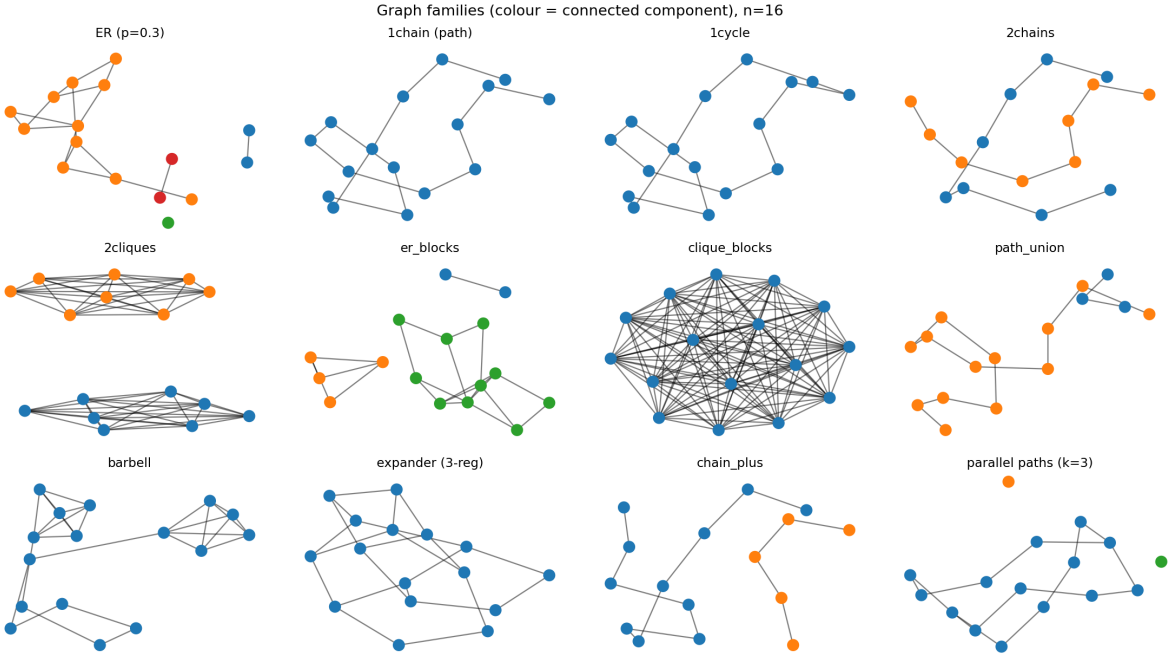

Figure 1: One example of each family ($n = 16$; node colour = connected component).

3 Thread 1: what makes a graph hard — the diameter and the density

We want to know what makes a graph hard for the standard model (the plain linear read-out). The metric throughout is **exact-match**: for a graph on n nodes the model outputs an $n \times n$ matrix of yes/no answers ("is node i connected to node j?"), and exact-match $= 1$ only if that whole matrix is right. We look at three candidate explanations in turn — the diameter, the density, the spectral gap — using the standard model trained on ER$(20, 0.08)$ (and the matching $n = 40$ model), evaluated on the families of Section 2.1.

3.1 How accuracy depends on distance: the capacity wall

The cleanest way to see the diameter's effect is to feed the model a long chain — a graph that contains node pairs at *every* distance — and look at how often it gets a pair right as a function of how far apart the two nodes are. Figure 2 does this. Accuracy is essentially perfect up to distance ≈ 9 and falls beyond it. The reason is the result from Report III: a 2-layer model can confirm a connection only within $3^L = 3^2 = 9$ hops; past that it sees no short path and can no longer tell a *connected-but-far* pair from a *disconnected* one, so it starts to make mistakes. (The partial recovery on the very farthest pairs is a thing we do not fully explain and do not rely on.) This is the main way the diameter matters: a graph whose farthest pairs sit beyond 9 will have wrong entries, so its exact-match drops. Figure 2 is an average over seeds; the per-seed numbers behind it (which seeds wall where, and which recover) are given later in Section 3.5.

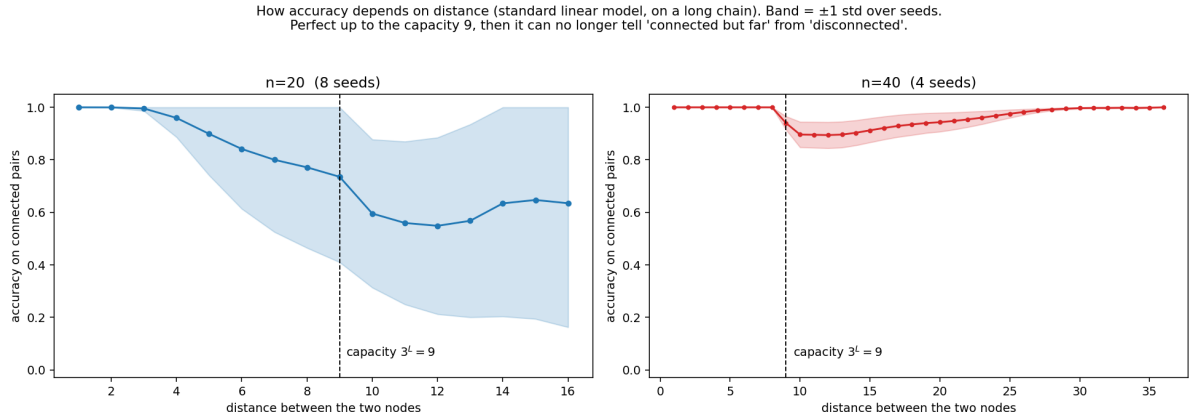

Figure 2: Accuracy on connected pairs vs the shortest-path distance between the two nodes. *Model:* the standard transformer ($L = 2$, $d_{\text{model}} = 512$, single head, *linear* read-out) trained on *ER only* — ER$(20, 0.08)$ at $n = 20$ (8 seeds, left) and ER$(40, 0.05)$ at $n = 40$ (4 seeds, right). *Test set:* the held-out **chain_plus** family (a long path plus a small separate component), which the model never saw in training; we use it because it contains pairs at every distance. Each line is the mean over the seeds, the band ±1 std. Perfect up to the capacity $3^L = 9$, then a drop; the $n = 20$ band is wide because of the seed lottery (per-seed in Section 3.5), the $n = 40$ curves are tighter.

3.2 Density also matters, and it is a separate effect

Distance is not the only thing. The model was trained only on *sparse* graphs (an ER$(20, 0.08)$ graph has on average ≈ 1.6 neighbours per node). If we hand it a much *denser* graph it fails even when the diameter is tiny, simply because such a graph is unlike anything it saw in training (out-of-distribution). We measure density by the **mean degree** (the average number of neighbours per node). Figure 3 holds the diameter small (so the capacity wall is out of the picture) and varies the density: accuracy is high near the training density and falls as the graph gets denser, very sharply at $n = 40$ (which was trained even sparser). So difficulty has (at least) two separate ingredients: *distance vs. the capacity* and *density vs. the training set*.

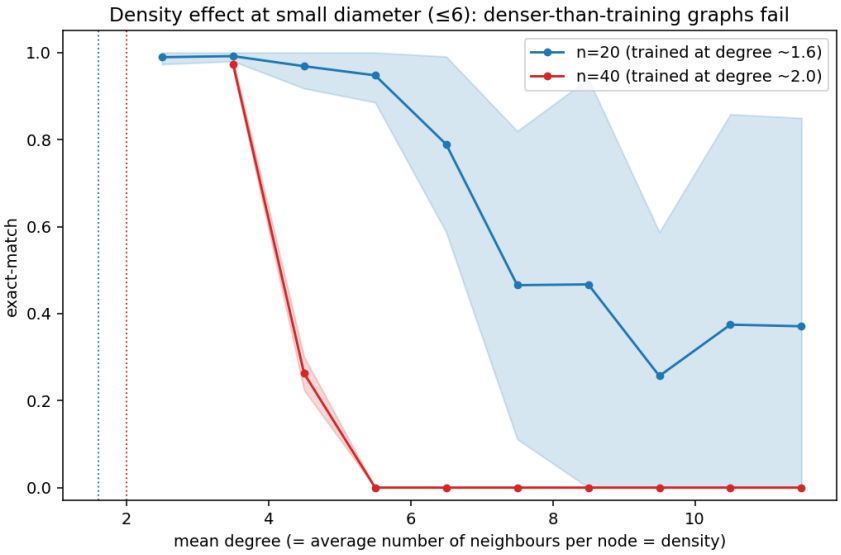

Figure 3: Whole-graph exact-match vs mean degree (density). *Same model as Figure 2* (standard, linear read-out, ER-trained; $n = 20$ over 8 seeds, $n = 40$ over 4 seeds, mean ± 1 std). *Test set:* a purpose-built pool of single-component graphs (the difficulty-map pool: an ER density sweep, a random-regular degree sweep, barbells, chains and cycles), here restricted to small diameter (≤ 6) so the density effect is isolated from the capacity wall. Both models do well near their training density (dotted lines) and fail on denser graphs; the sparser-trained $n = 40$ model collapses almost immediately.

3.3 The spectral gap: we cannot separate it from the diameter

The third candidate is the spectral gap (a small gap = a bottleneck). The problem is that, for ordinary sparse graphs, the gap is essentially just *another name for the diameter*: a long, thin graph has a large diameter *and* a small gap, while a compact graph has a small diameter *and* a large gap. Figure 4 plots one dot per graph (diameter on the x-axis, gap on the y-axis): the dots fall on a single diagonal line — they are collinear, correlation -0.82 — with no graphs in the "large diameter and large gap" region. Because the two move together, no experiment on these graphs can say whether the difficulty is caused by the diameter or by the gap.

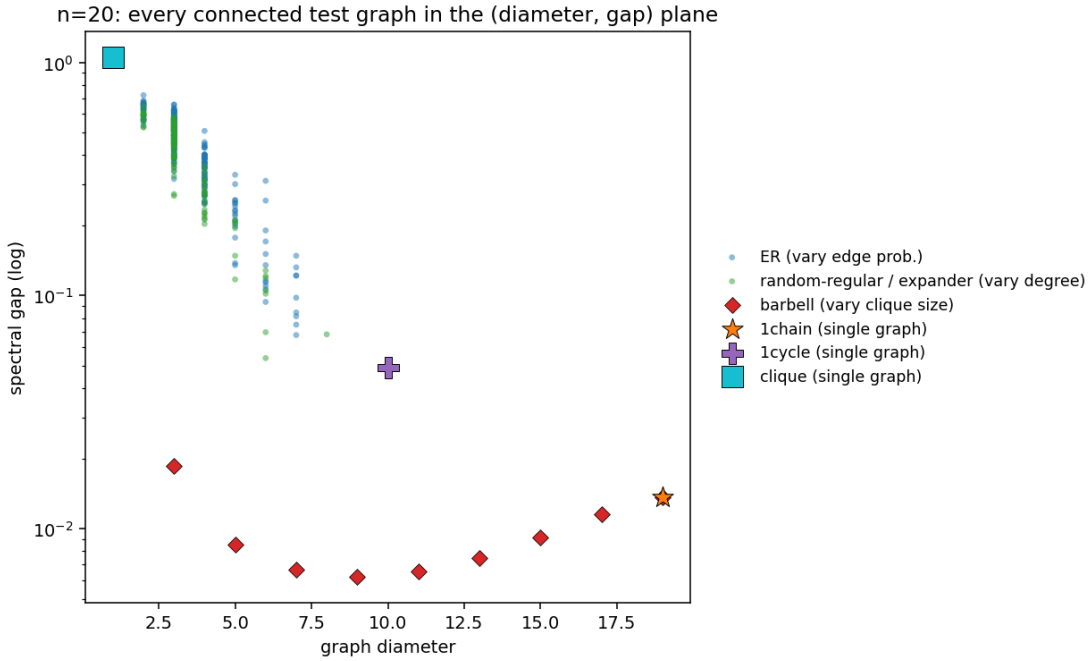

Figure 4: Where the test graphs sit in the (diameter, gap) plane ($n = 20$; $n = 40$ looks the same, correlation -0.82). Only *connected* graphs are shown, because the diameter and the spectral gap are only defined for a single component — the disconnected families are made of these same pieces (a 2-chain is two paths, a 2-clique two cliques, a 2-cycle two cycles). ER, random-regular/expander and barbell are each swept over a parameter (edge probability, degree, clique size), so they form clouds; 1chain, 1cycle and a single clique are one fixed graph each at $n = 20$, hence one marker apiece. Everything except the barbell lies on one descending diagonal ridge (clique → ER/expander → 1cycle → 1chain): a larger diameter comes with a smaller gap, with nothing in the "large diameter *and* large gap" corner, so the two cannot be told apart. Only the barbell (red) sits below the ridge — a moderate diameter at a tiny gap — and it gets there only by being locally dense (its two cliques).

The collinearity has a single exception: the **barbell** (two dense cliques joined by one thin bridge), the red dots sitting *below* the ridge in Figure 4 — a moderate diameter at a tiny gap. It is the one graph that lets us ask whether a bottleneck (a small gap) makes a graph hard *beyond* what its diameter says. That question, and the rest of the spectral-gap analysis, is the subject of Thread 2 (Section 4); we keep it out of this thread, which is about the two axes we *can* separate.

3.4 Summary: the family panel

Table 1 collects the whole picture on the families. We trained *eight* copies of the baseline model (eight random seeds, ER training, linear read-out) and evaluated each on every family; the table reports, per family, its *typical* (median) diameter and gap — a label for where that family sits — the mean exact-match over the eight models, and the standard deviation of that exact-match across the eight seeds ("seed sd"). How to read it: sorting by gap, the families with a small gap / large diameter (top of the table) are hard and the well-mixed ones (ER, expander) are easy, which is the "gap matters" impression — but, as above, that is mostly the diameter and the density in disguise. The large *seed sd* on every structured family (≥ 0.35) is its own warning: per seed the exact-match is essentially all-or-nothing (a model either finds the generalising solution or does not, the seed lottery of Report III), so the mean is a difficulty *ranking* over the lucky seeds, not a number any single run will hit. Only ER and expander are

seed-stable (sd ≤ 0.03).

Family	typ. diameter	typ. gap	exact match	seed sd
barbell	11	0.007	0.22	0.35
1chain	19	0.014	0.45	0.46
1cycle	10	0.049	0.58	0.46
2chains	9	0.060	0.10	0.18
ER(0.08)	6	0.108	0.98	0.03
expander	5	0.153	**0.99**	0.01
2cliques	1	1.11	0.38	0.46

Table 1: Baseline model (ER-trained, linear read-out, $n = 20$) evaluated on each family. "typ. diameter / gap" is the median diameter / spectral gap of graphs in that family; "exact match" is the fraction of graphs predicted entirely correctly, averaged over the 8 training seeds; "seed sd" is the standard deviation of that fraction across the 8 seeds. Rows sorted by gap.

3.5 Per-seed detail by node-pair distance: where the wall starts, where it recovers

A clarification on what these tables measure. They are about a single *pair* of nodes: each entry is the accuracy on the node pairs whose shortest-path distance is exactly d (only connected pairs, i.e. "reach"), *not* a whole-graph score. So they answer "can the model tell that two nodes d hops apart are connected?". (The complementary whole-graph view — exact-match as a function of the graph's diameter — is in Section 3.6 below.) Figure 2 averaged this over seeds; the per-seed numbers behind it are more telling, and let us compare the two training sets directly. Tables 2–4 give it for the three families with long within-component distances, for all eight ER-trained seeds and (last row) the mixed model.

ER-trained: a seed-dependent wall, with a recovery on some seeds. For the seeds that found the generalising solution, accuracy is ≈ 1 up to the capacity 9, dips just past it, and then *climbs back* on the farthest pairs — the recovery. On chain_plus, seed 1000 falls to 0.72 at $d = 11$ and returns to 1.00 by $d = 15$; seed 3000 never dips at all (1.00 throughout). The seeds that did *not* generalise $(2000, 4000, 5000)$ instead slide down monotonically and never recover (seed 2000 reaches 0.07 at $d = 16$). This is the seed lottery of Table 1 seen distance-by-distance. The recovery is also family-dependent: it is strong on chain_plus (back to 1.00), weaker on 1chain (seed 3000 bottoms at 0.69 around $d = 13$ and only reaches 0.98 by $d = 16$).

Mixed-trained: the wall is gone here. The mixed model solves all three families: ≈ 1.00 at every distance and every seed (last row of each table). Two of them (1chain, path_union) are in its training stream; chain_plus is held out, but it is built from the same chain pieces, so the model handles it too. The wall being absent at $n = 20$ is *not* because the model is now unbounded — it is because at $n = 20$ the chains are too short to contain distances far enough past 9 to expose it. The $n = 40$ tables below, where the chains are long enough, show the wall return even on families the model was trained on.

seed	$d5$	$d7$	$d9$	$d10$	$d11$	$d12$	$d13$	$d14$	$d15$	$d16$
ER-trained										
1000	1.00	1.00	0.99	0.73	0.72	0.73	0.80	0.96	1.00	1.00
2000	0.53	0.26	0.16	0.13	0.10	0.08	0.07	0.06	0.06	0.07
3000	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
4000	0.88	0.63	0.42	0.29	0.17	0.11	0.09	0.07	0.04	0.00
5000	0.77	0.51	0.39	0.34	0.26	0.20	0.15	0.11	0.09	0.01
6000	1.00	1.00	0.98	0.76	0.73	0.74	0.79	0.93	0.99	1.00
7000	1.00	1.00	0.93	0.72	0.72	0.75	0.82	0.97	1.00	1.00
8000	1.00	1.00	1.00	0.80	0.77	0.78	0.83	0.98	1.00	1.00
mixed (all 8)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Table 2: **chain_plus**: accuracy on connected pairs by distance d, $n = 20$, linear read-out. Top: eight ER-trained seeds (the wall at 9 + the recovery, both seed-dependent). Bottom: the mixed-trained model is ≈ 1.00 everywhere (all eight seeds identical).

seed	$d5$	$d7$	$d9$	$d10$	$d11$	$d12$	$d13$	$d14$	$d15$	$d16$
ER-trained										
1000	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2000	0.55	0.37	0.31	0.30	0.29	0.30	0.29	0.25	0.22	0.18
3000	1.00	1.00	0.88	0.81	0.76	0.71	0.69	0.72	0.85	0.98
4000	0.87	0.78	0.75	0.75	0.74	0.73	0.71	0.68	0.63	0.57
5000	0.77	0.61	0.57	0.56	0.54	0.53	0.51	0.47	0.42	0.34
6000	1.00	1.00	1.00	0.99	0.99	0.99	0.99	0.99	0.98	0.97
7000	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
8000	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
mixed (all 8)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Table 3: **1chain** (a single path over all nodes), same layout as Table 2. The recovery is weaker here than on chain_plus.

seed	$d5$	$d7$	$d9$	$d10$	$d11$	$d12$	$d13$	$d14$	$d15$	$d16$
ER-trained										
1000	1.00	1.00	0.98	0.91	0.93	0.96	0.98	1.00	1.00	1.00
2000	0.64	0.38	0.31	0.30	0.31	0.31	0.32	0.30	0.28	0.26
3000	1.00	1.00	0.93	0.88	0.84	0.78	0.75	0.77	0.87	0.99
4000	0.87	0.69	0.62	0.62	0.62	0.63	0.63	0.64	0.62	0.59
5000	0.79	0.58	0.54	0.54	0.54	0.53	0.52	0.49	0.45	0.39
6000	1.00	1.00	0.97	0.90	0.93	0.95	0.97	0.99	0.99	0.98
7000	1.00	1.00	0.95	0.90	0.93	0.96	0.98	1.00	1.00	1.00
8000	1.00	1.00	0.99	0.92	0.94	0.96	0.98	1.00	1.00	1.00
mixed (all 8)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Table 4: **path_union** (a disjoint union of 1–4 random paths), same layout.

The same families at $n = 40$, where the capacity is clearly visible. Tables 5–6 repeat this at $n = 40$, where the chains reach distance 36 and the columns are placed finely around 9 so the wall is easy to read. Two things change. First, it is now the *mixed* model that shows the wall in its sharpest form: exactly 1.00 up to $d = 9$ and then a step down to ≈ 0.55 at $d = 10$ — the capacity $3^L = 9$ read off directly — followed by the recovery on the farthest

pairs. (The mixed model was perfect on these families at $n = 20$ only because $n = 20$ could not hold distances past the capacity; at $n = 40$ it can, and the wall appears.) Second, the ER-trained model is uniformly mediocre instead ($\approx$ 0.85–0.97 at all distances, no step at 9): never having seen the structure, it is partly right everywhere rather than perfect-then-broken. (1chain is the one family with no wall at all for the mixed model — it stays 1.00 at every distance, because a single connected component can be predicted correctly by simply answering "connected" everywhere, with no long-range reach needed; we therefore drop its table. It is the families requiring a genuine *cut*, chain_plus and path_union, that expose the capacity.)

seed	*d*7	*d*8	*d*9	*d*10	*d*11	*d*12	*d*14	*d*18	*d*24	*d*30	*d*36
ER-trained											
1000	1.00	1.00	0.91	0.83	0.82	0.82	0.83	0.88	0.96	1.00	1.00
2000	1.00	1.00	0.93	0.88	0.88	0.87	0.88	0.91	0.95	1.00	1.00
3000	1.00	1.00	0.96	0.94	0.94	0.93	0.94	0.96	0.97	0.99	1.00
4000	1.00	1.00	0.97	0.95	0.95	0.95	0.96	0.99	1.00	1.00	1.00
mixed-trained											
1000	1.00	1.00	**1.00**	**0.56**	0.58	0.59	0.59	0.56	0.75	0.92	0.99
2000	1.00	1.00	**1.00**	**0.52**	0.53	0.54	0.54	0.51	0.71	0.89	0.91
3000	1.00	1.00	**1.00**	**0.57**	0.59	0.60	0.58	0.52	0.70	0.89	1.00
4000	1.00	1.00	**1.00**	**0.52**	0.53	0.54	0.54	0.51	0.72	0.89	0.99

Table 5: **chain_plus**, $n = 40$, linear read-out, accuracy on connected pairs by distance, four seeds each. The mixed model is exactly 1.00 up to the capacity 9 and steps down at 10 (bold) — the $3^L = 9$ wall read directly — then recovers far out. The ER-trained model has no such step; it is fuzzy at every distance.

seed	*d*7	*d*8	*d*9	*d*10	*d*11	*d*12	*d*14	*d*18	*d*24	*d*30	*d*36
ER-trained											
1000	1.00	1.00	0.88	0.80	0.81	0.81	0.83	0.86	0.85	0.79	0.98
2000	1.00	1.00	0.91	0.84	0.85	0.85	0.87	0.90	0.91	0.87	0.96
3000	1.00	1.00	0.93	0.87	0.87	0.88	0.89	0.90	0.87	0.79	0.87
4000	1.00	1.00	0.96	0.92	0.93	0.93	0.95	0.98	0.98	0.98	1.00
mixed-trained											
1000	1.00	1.00	**1.00**	**0.63**	0.65	0.66	0.70	0.77	0.92	0.99	1.00
2000	1.00	1.00	**1.00**	**0.61**	0.63	0.64	0.68	0.75	0.91	0.98	1.00
3000	1.00	1.00	**1.00**	**0.64**	0.66	0.67	0.70	0.76	0.91	0.98	1.00
4000	1.00	1.00	**1.00**	**0.61**	0.63	0.64	0.68	0.75	0.91	0.99	1.00

Table 6: **path_union**, $n = 40$. Same story as chain_plus: the mixed model steps down exactly at the capacity 9 (bold) and recovers; the ER model is fuzzy throughout.

An even sharper wall: 2chains and 1cycle. chain_plus and path_union drop to a ≈ 0.55 floor past the capacity and then recover. Two other families show the wall in its purest form. On 2chains and 1cycle (Tables 7–8, $n = 40$; these only become long enough to test past 9 at $n = 40$) the mixed model is exactly 1.00 up to $d = 9$ and *exactly* 0.00 from $d = 10$ on — a clean step right at the capacity, with no floor and no recovery, identical across all four seeds. These are simple repetitive structures (two paths, one ring), so beyond 9 hops the model has nothing to fall back on. Crucially, 2chains and 1cycle *are in the mixed training stream*, and each is a single graph (up to a node permutation) that the model sees on the order of 10^8 times — yet it still cannot connect a pair more than 9 hops apart. That makes this the cleanest evidence that the wall is an *architectural* limit of the $L = 2$ model ($3^L = 9$), not a gap in the training

data. The ER-trained model, which never saw them, is by contrast fuzzy and seed-dependent (one ER seed even solves 2chains, another nearly collapses on 1cycle). So the capacity is the same 9 across all these families; what differs is the behaviour *past* it — a floor-and-recovery (chain_plus, path_union) or a drop straight to zero (2chains, 1cycle, and likewise 2cycle).

seed	$d8$	$d9$	$d10$	$d12$	$d14$	$d16$	$d18$	$d19$
ER-trained								
1000	1.00	0.88	0.75	0.77	0.82	0.91	0.95	1.00
2000	1.00	0.93	0.79	0.77	0.76	0.83	0.89	0.99
3000	1.00	0.96	0.89	0.87	0.88	0.91	0.95	0.99
4000	1.00	0.99	0.99	1.00	1.00	1.00	1.00	1.00
mixed-trained								
1000	1.00	**1.00**	**0.00**	0.04	0.04	0.01	0.00	0.00
2000	1.00	**1.00**	**0.00**	0.02	0.01	0.00	0.00	0.00
3000	1.00	**1.00**	**0.00**	0.00	0.00	0.00	0.00	0.00
4000	1.00	**1.00**	**0.00**	0.00	0.00	0.00	0.00	0.00

Table 7: **2chains**, $n = 40$ (two disjoint paths of 20 nodes). The mixed model is a clean step: 1.00 to the capacity 9, then 0.00 (bold), no recovery. The ER model is fuzzy, and seed 4000 happens to solve it.

seed	$d8$	$d9$	$d10$	$d12$	$d14$	$d16$	$d18$	$d20$
ER-trained								
1000	0.91	0.24	0.14	0.14	0.14	0.14	0.14	0.14
2000	0.96	0.65	0.70	0.70	0.70	0.70	0.70	0.70
3000	0.98	0.52	0.54	0.54	0.54	0.54	0.54	0.54
4000	0.98	0.77	0.78	0.78	0.78	0.78	0.78	0.78
mixed-trained								
1000	1.00	**1.00**	**0.00**	0.00	0.00	0.00	0.00	0.00
2000	1.00	**1.00**	**0.00**	0.00	0.00	0.00	0.00	0.00
3000	1.00	**1.00**	**0.00**	0.00	0.00	0.00	0.00	0.00
4000	1.00	**1.00**	**0.00**	0.00	0.00	0.00	0.00	0.00

Table 8: **1cycle**, $n = 40$ (one ring C_{40}). Same clean step at 9 for the mixed model. The ER model sits at a seed-dependent floor (0.14–0.78).

3.6 Per-seed detail by graph diameter

The tables above scored individual node pairs. The complementary question is at the *whole-graph* level: as a graph's *diameter* grows, how often is its *entire* connectivity matrix predicted correctly (exact-match)? A graph is exact-match = 1 only if every one of its pairs is right, so a graph whose diameter exceeds the capacity necessarily contains at least one pair beyond reach and fails. We show this only for the families where the diameter *varies* (the fixed-diameter families — 2chains, 1cycle, the cliques, barbell, . . . — are a single point and uninformative here), and only the interesting ones: **path_union** (a sparse family whose diameter is the length of its longest path, so it is a clean diameter sweep at fixed density) and **ER** (the training distribution).

path_union. Tables 9–10 show the graph-level wall directly. For the mixed model at $n = 20$, exact-match is ≈ 1 for diameter up to 9, drops over diameters 10–13 (the graph now contains pairs just past the capacity), and *recovers* to 1.00 by diameter 16 (the same boundary recovery, now at the graph level). The ER-trained model shows the same shape on the seeds that

generalise (e.g. seed 1000: 1.00 at diameter 8, 0.28 at 10, back to 0.99 at 16) and is flat at 0 on the seeds that do not — the seed lottery again. At $n = 40$ the wall is wider (diameter 10–33 all near 0) with the recovery only on the very largest diameters.

ER (training distribution). Table 11 is the in-distribution counterpart. The mixed model solves ER at every diameter it produces (≤ 13 at $n = 20$, all ≈ 1). The ER-trained model again splits by seed: the generalising seeds are 1.00 throughout, the others decay as the diameter grows (seed 2000: 0.89 at diameter 9, 0.00 at 13) — a larger-diameter ER graph contains longer pairs, so the non-algorithmic seeds drop.

seed	$D6$	$D8$	$D9$	$D10$	$D11$	$D13$	$D16$	$D19$
ER-trained								
1000	1.00	1.00	0.81	0.28	0.31	0.40	0.99	0.94
2000	0.33	0.01	0.00	0.00	0.00	0.00	0.00	0.00
3000	0.99	0.43	0.27	0.31	0.40	0.46	0.71	0.00
4000	0.22	0.00	0.00	0.00	0.00	0.03	0.00	0.00
5000	0.14	0.00	0.00	0.00	0.00	0.02	0.02	0.00
6000	1.00	1.00	0.71	0.26	0.29	0.41	1.00	0.69
7000	1.00	1.00	0.80	0.21	0.30	0.42	0.97	0.98
8000	1.00	0.98	0.78	0.26	0.31	0.42	1.00	0.96
mixed-trained								
1000	1.00	1.00	0.98	0.87	0.63	0.63	1.00	1.00
2000	1.00	1.00	0.99	0.96	0.80	0.65	1.00	1.00
3000	1.00	1.00	0.99	0.92	0.71	0.65	1.00	1.00
4000	1.00	1.00	0.99	0.93	0.68	0.61	1.00	1.00
5000	1.00	1.00	1.00	0.91	0.75	0.63	0.99	1.00
6000	1.00	1.00	0.99	0.92	0.68	0.66	1.00	1.00
7000	1.00	1.00	1.00	0.89	0.71	0.66	1.00	1.00
8000	1.00	1.00	0.98	0.89	0.71	0.64	1.00	1.00

Table 9: **path_union**, $n = 20$: *whole-graph* exact-match as a function of the graph's diameter, per seed. Mixed: 1.00 to diameter 9, a dip over 10–13, recovery by 16. ER-trained: the same shape on the generalising seeds, flat 0 on the rest.

seed	$D10$	$D14$	$D20$	$D28$	$D33$	$D35$	$D37$	$D39$
ER-trained								
1000	0.00	0.00	0.00	0.03	0.23	0.78	0.03	0.00
2000	0.00	0.00	0.00	0.02	0.22	0.82	0.17	0.00
3000	0.00	0.00	0.00	0.01	0.16	0.71	0.03	0.00
4000	0.00	0.00	0.00	0.01	0.15	0.68	0.67	0.09
mixed-trained								
1000	0.00	0.00	0.00	0.00	0.00	0.01	0.03	1.00
2000	0.00	0.00	0.00	0.00	0.01	0.00	0.07	1.00
3000	0.00	0.00	0.00	0.02	0.10	0.70	0.86	1.00
4000	0.00	0.00	0.00	0.00	0.03	0.07	0.09	1.00

Table 10: **path_union**, $n = 40$: whole-graph exact-match by graph diameter, per seed. The wall is wide (near 0 from diameter 10 to ~ 33); the recovery appears only at the largest diameters (a single 39-node path, which the mixed model gets entirely right).

seed	$D6$	$D8$	$D9$	$D10$	$D11$	$D12$	$D13$
ER-trained							
1000	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2000	0.95	0.95	0.89	0.77	0.54	0.27	0.00
3000	1.00	1.00	0.99	1.00	1.00	1.00	1.00
4000	0.95	0.94	0.90	0.84	0.60	0.59	0.00
5000	0.95	0.95	0.89	0.81	0.60	0.55	1.00
6000	1.00	1.00	1.00	0.99	1.00	1.00	1.00
7000	1.00	1.00	1.00	1.00	1.00	1.00	1.00
8000	1.00	1.00	1.00	1.00	1.00	1.00	1.00
mixed-trained							
1000	1.00	1.00	0.99	0.99	0.96	1.00	1.00
2000	1.00	1.00	0.99	0.99	0.96	1.00	1.00
3000	1.00	1.00	0.99	0.99	0.96	1.00	1.00
4000	1.00	1.00	0.99	0.99	0.94	1.00	1.00
5000	1.00	1.00	0.99	0.98	0.96	1.00	1.00
6000	1.00	1.00	0.99	0.99	0.96	1.00	1.00
7000	1.00	1.00	0.99	0.99	0.96	1.00	1.00
8000	1.00	1.00	0.99	0.99	0.94	1.00	1.00

Table 11: $\mathbf{ER}(20, 0.08)$ (the training distribution), whole-graph exact-match by graph diameter, per seed. The mixed model solves every diameter; the ER-trained model splits by seed (generalising seeds flat at 1.00, the rest decaying as the diameter grows).

4 Thread 2: the spectral gap — does a bottleneck add difficulty?

Thread 1 found two difficulty axes we can separate (distance-vs-capacity and density-vs-training) and showed that on ordinary graphs the spectral gap is just collinear with the diameter (Figure 4), so it says nothing new there. The one exception was the barbell — a real bottleneck (tiny gap) at a moderate diameter. This thread asks the gap question directly: when a graph really does have a bottleneck, does that make it hard *beyond* what its diameter predicts, and can we measure the gap's effect on its own? We try three constructions. Short answer: a bottleneck does add difficulty (robustly, as a direction), but no construction isolates the gap cleanly. As in Thread 1, every table keeps the ER-trained and mixed-trained models separate.

4.1 The barbell: a bottleneck fails deep inside the capacity

The barbell is two dense cliques joined by one thin bridge: one connected component, a moderate diameter, and the smallest gap of all our families. It is held out of training, so it probes a bottleneck the model never saw. Tables 12–13 give its accuracy on connected pairs by distance, per seed.

The clean result is at $n = 40$ (Table 12): the mixed model gets the within-clique pairs right ($d = 1$: 0.99) but *collapses at* $d = 3$ (≈ 0.4) — four times *inside* the capacity 9, where every non-bottleneck family is still perfect at $d \leq 9$. The pairs that fail are exactly those whose only route crosses the bridge, and all four seeds agree. (The ER-only model never saw cliques, so it is poor on the barbell at every distance, including $d = 1$.) A simple path also has a small spectral gap, but the model handles it up to distance 9. So the barbell fails not just because its spectral gap is small, but because it contains a narrow cut: one single bridge separating two dense regions. At $n = 20$ (Table 13) the cliques are larger relative to n, so the bridge sits further out and the collapse is at $d \approx 7$–9 rather than 3; the effect is the same, only at a different distance, and the ER model shows the usual seed lottery.

seed	$d1$	$d2$	$d3$	$d5$	$d7$	$d9$	$d11$	$d13$	$d15$
ER-trained									
1000	0.35	0.45	0.39	0.35	0.36	0.26	0.18	0.12	0.10
2000	0.38	0.43	0.42	0.40	0.42	0.22	0.12	0.08	0.07
3000	0.24	0.56	0.34	0.30	0.34	0.21	0.14	0.08	0.04
4000	0.15	0.35	0.33	0.28	0.25	0.19	0.05	0.01	0.00
mixed-trained									
1000	0.99	0.75	**0.38**	0.59	0.56	0.51	0.42	0.15	0.00
2000	0.99	0.57	**0.43**	0.62	0.57	0.54	0.39	0.13	0.00
3000	0.99	0.92	**0.39**	0.60	0.58	0.53	0.51	0.38	0.00
4000	0.99	0.81	**0.45**	0.61	0.57	0.53	0.44	0.11	0.00

Table 12: **barbell**, $n = 40$: accuracy on connected pairs by distance, per seed. The mixed model is fine within a clique ($d = 1$) but collapses at $d = 3$ (bold) — the bridge — far inside the capacity 9. The ER-only model never saw cliques and fails throughout.

seed	$d1$	$d2$	$d3$	$d5$	$d7$	$d9$	$d11$
ER-trained							
1000	1.00	1.00	1.00	0.69	0.76	0.99	0.12
2000	1.00	0.98	0.91	0.88	0.85	1.00	1.00
3000	1.00	1.00	0.84	0.47	0.38	0.45	0.99
4000	1.00	0.99	0.99	0.99	0.99	1.00	1.00
5000	1.00	1.00	1.00	1.00	1.00	1.00	1.00
6000	1.00	1.00	1.00	0.68	0.68	0.96	0.16
7000	1.00	1.00	1.00	0.73	0.86	0.98	0.02
8000	1.00	1.00	1.00	0.94	0.97	0.99	0.75
mixed-trained							
1000	1.00	1.00	1.00	1.00	0.62	0.04	0.00
2000	1.00	1.00	1.00	1.00	0.69	0.09	0.00
3000	1.00	1.00	1.00	1.00	0.61	0.01	0.00
4000	1.00	1.00	1.00	1.00	0.67	0.10	0.00
5000	1.00	1.00	1.00	1.00	0.64	0.02	0.00
6000	1.00	1.00	1.00	1.00	0.71	0.04	0.00
7000	1.00	1.00	1.00	1.00	0.68	0.01	0.00
8000	1.00	1.00	1.00	1.00	0.66	0.02	0.00

Table 13: **barbell**, $n = 20$: same. The cliques are larger relative to n, so the bridge collapse is at $d \approx 7$–9; the mixed model is consistent, the ER model shows the seed lottery.

4.2 barbell_var: a sweep that turns out not to be about the gap

To grade the effect, *barbell_var* varies the clique size (and so the bridge length) while keeping the barbell shape. We sweep the full range (cliques from 2 up to $n/2$), but on closer inspection this does *not* isolate the gap, for two reasons. First, a barbell's gap is intrinsically tiny and barely moves: across the whole sweep it stays in ≈ 0.006–0.019 at $n = 20$ and ≈ 0.0008–0.005 at $n = 40$, and it is *non-monotone* (U-shaped) in the clique size — larger at both ends (a long thin bridge, or a short bridge between big cliques) and smallest in between. So there is no wide, monotone gap range to read off. Second, changing the clique size changes the diameter ($3 \to 19$ at $n = 20$) and the clique *density* at the same time.

Looking at the accuracy by the actual knob (Tables 14–15, exact-match by diameter) makes clear that the difficulty tracks those, not the gap. The model fails on the big-dense-clique barbells (small diameter: two 10-cliques are dense, out-of-distribution, and trigger the degree

heuristic of Section 5.5) and succeeds only as the cliques shrink to sparse 2–4-node ends on a long bridge (large diameter) — while the gap is essentially constant across that whole range. So barbell_var, like expander_var below, does not isolate the gap: it just reconfirms that density and the bridge length matter (Thread 1). (The read-out makes no difference to it either — linear and similarity are identical — Thread 3.)

seed	$D3$	$D7$	$D11$	$D15$	$D19$
ER-trained					
1000	0.00	0.00	0.00	1.00	0.94
2000	0.00	0.00	0.03	1.00	0.00
3000	0.00	0.00	0.00	0.05	0.00
4000	0.00	0.00	0.74	0.97	0.00
5000	0.00	0.00	0.90	1.00	0.00
6000	0.00	0.00	0.00	1.00	0.68
7000	0.00	0.00	0.00	1.00	0.98
8000	0.00	0.00	0.07	1.00	0.99
mixed-trained					
1000	0.00	0.00	0.00	0.86	1.00
2000	0.00	0.00	0.00	0.84	1.00
3000	0.00	0.00	0.00	0.17	1.00
4000	0.00	0.00	0.00	0.52	1.00
5000	0.00	0.00	0.00	0.94	1.00
6000	0.00	0.00	0.00	0.90	1.00
7000	0.00	0.00	0.00	0.91	1.00
8000	0.00	0.00	0.00	0.68	1.00

Table 14: **barbell_var**, $n = 20$: exact-match by graph diameter, per seed. Diameter grows as the cliques *shrink* (the bridge lengthens): $D3$ is two 10-cliques, $D19$ two 2-cliques on a long thin bridge. The spectral gap stays ≈ 0.006–0.019 across the whole row, so this is not the gap — it is the clique density (big dense cliques fail) and the bridge.

seed	$D3$	$D11$	$D19$	$D27$	$D35$	$D39$
ER-trained						
1000	0.00	0.00	0.00	0.00	0.00	0.00
2000	0.00	0.00	0.00	0.00	0.02	0.00
3000	0.00	0.00	0.00	0.00	0.01	0.00
4000	0.00	0.00	0.00	0.00	0.10	0.07
mixed-trained						
1000	0.00	0.00	0.00	0.00	0.00	1.00
2000	0.00	0.00	0.00	0.00	0.00	1.00
3000	0.00	0.00	0.00	0.00	0.00	1.00
4000	0.00	0.00	0.00	0.00	0.00	1.00

Table 15: **barbell_var**, $n = 40$: same (gap ≈ 0.0008–0.005 throughout). Only the tiniest-clique, longest-bridge case ($D39$, essentially a long sparse path) is ever solved.

4.3 expander_var: a confounded negative

A natural way to sweep the gap continuously is to vary the degree of a random-regular graph (*expander_var*). The catch: a higher degree also means more edges — a denser graph — which is out-of-distribution for a model trained on sparse graphs. Tables 16–17 show this per seed. At $n = 20$ (Table 16) the mixed model solves every gap; the ER model is non-monotone and

strongly seed-dependent — the rise at small gaps is a real gap effect, but the fall at the largest gaps is the *density* confound (the densest, most out-of-distribution graphs), not the gap. At $n = 40$ (Table 17) the same shape is even sharper and now consistent across seeds: the ER model collapses already by gap 0.3 (rather than 0.6 at $n = 20$), because the $n = 40$ model was trained on the sparser ER$(40, 0.05)$ and tolerates even less density. So at both sizes, varying the degree changes two things at once and does not isolate the gap.

seed	0.04–0.08	0.08–0.15	0.15–0.3	0.3–0.6	0.6–1.0
ER-trained					
1000	1.00	1.00	1.00	0.53	0.00
2000	0.05	0.96	0.95	0.99	1.00
3000	0.59	1.00	1.00	0.72	0.01
4000	0.05	0.96	0.99	0.90	0.99
5000	0.05	0.93	0.96	0.80	0.99
6000	1.00	1.00	1.00	0.83	0.09
7000	1.00	1.00	1.00	0.59	0.01
8000	1.00	1.00	1.00	1.00	0.96
mixed (all 8)	1.00	1.00	1.00	1.00	1.00

Table 16: **expander_var**, $n = 20$: exact-match by spectral-gap bucket, per seed (higher gap = higher degree = denser). The mixed model solves all; the ER model's rise at small gaps is a real gap effect, its fall at large gaps the density confound.

seed	0.04–0.08	0.08–0.15	0.15–0.3	0.3–0.6	0.6–1.0
ER-trained					
1000	0.97	0.99	0.47	0.00	0.00
2000	0.84	0.71	0.53	0.00	0.00
3000	0.87	0.88	0.78	0.01	0.00
4000	0.97	0.97	0.76	0.01	0.00
mixed-trained					
1000	0.97	1.00	1.00	1.00	1.00
2000	1.00	1.00	1.00	1.00	1.00
3000	1.00	1.00	1.00	1.00	1.00
4000	1.00	0.94	1.00	1.00	1.00

Table 17: **expander_var**, $n = 40$: same buckets. The ER model's density collapse is sharper than at $n = 20$ (it reaches 0 already by gap 0.3) and consistent across seeds. The mixed model solves these buckets; its only failure (not shown, a single degree-2 graph at gap < 0.04) is the cycle C_{40} of diameter 20 — the capacity wall of Thread 1, not a gap effect.

4.4 A confound-free probe: the bottleneck *can* be isolated, and it matters

The two sweeps above did not isolate the gap because each moved something else with it. So we built a probe that holds everything else fixed. Two terminals s, t are joined by k internally-disjoint paths of length ℓ, so the *distance* $\mathrm{dist}(s, t) = \ell$ is the same for every k; the terminals are padded with leaves to a *fixed degree* (so they do not become higher-degree hubs as k grows); and the rest of the canvas is filled with a sparse path (so the *density* stays ≈ 2, like the training graphs, with no isolated padding). The only thing that changes with k is the number of parallel routes between the terminals, i.e. the effective resistance ℓ/k. We measure the accuracy on the (s, t) pair (always connected) at each (k, ℓ), on the standard *linear* models at $n = 40$, ER-trained and mixed-trained, four seeds each (Table 18, Figure 5).

Now the answer is clean. *Within* capacity ($\ell \leq 9$) the mixed model gets the pair right with a single path already, so k barely matters. *Beyond* capacity ($\ell = 11, 13, 15$) a single path ($k = 1$) fails (accuracy ≈ 0.55 — the distance is past $3^L = 9$), but adding parallel routes rescues it: $k = 2$ lifts it to ≈ 0.93 and $k = 3$ to 1.00, at the *same* distance and density. So the effective resistance genuinely matters, independently of the distance: with k routes there are k length-ℓ walks instead of one, the connection signal is k times stronger, and the model confirms a connection it could not confirm over a single path of the same length. (The ER model, which never saw path-like graphs, is noisy and shows no such structure.) This is the one place we *do* isolate the bottleneck, and the answer is positive: a tighter bottleneck (fewer routes, higher resistance) is harder, at fixed distance and density. It does not contradict Thread 1 — on *natural* graphs the gap is still collinear with the diameter, so it takes a built probe to separate them — but it shows the bottleneck is a real, separable cause, not merely a confounded direction.

	within capacity			beyond capacity		
k	ℓ=5	ℓ=7	ℓ=9	ℓ=11	ℓ=13	ℓ=15
ER-trained						
1	1.00	1.00	1.00	0.94	0.88	0.93
2	1.00	1.00	0.98	0.99	0.99	1.00
3	0.93	0.75	1.00	1.00	1.00	–
4	0.98	0.81	1.00	–	–	–
mixed-trained						
1	0.79	0.94	0.99	**0.55**	**0.53**	**0.55**
2	1.00	1.00	1.00	**0.93**	**0.93**	**0.95**
3	1.00	1.00	1.00	**1.00**	**1.00**	–
4	1.00	1.00	1.00	–	–	–

Table 18: Confound-free parallel-paths probe. *Model:* the standard *linear* read-out at $n = 40$, ER-trained (top block) and mixed-trained (bottom block). Each number is the *mean over the four training seeds* of the accuracy on the terminal (s, t) pair (always connected), measured on 400 freshly-sampled probe graphs per cell. Columns are the fixed distance ℓ; rows the number of parallel routes k (so resistance ℓ/k falls downward). Beyond the capacity ($\ell > 9$) a single route fails and more routes rescue it (bold) — the effective resistance matters at fixed distance. Empty cells do not fit in $n = 40$.

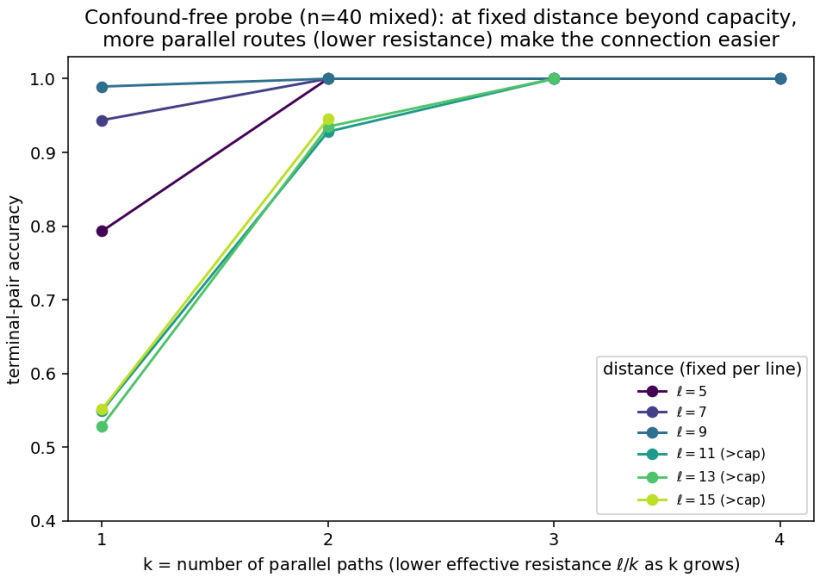

Figure 5: Table 18 for the mixed model. Each line is a fixed distance ℓ; the beyond-capacity lines ($\ell = 11, 13, 15$) climb steeply with k — at the same distance, more parallel routes (lower resistance) make the connection easy.

4.5 What we can and cannot say about the gap

A real bottleneck does add difficulty, and we can now say so two ways. At the pair level, the fixed barbell collapses on the bridge-crossing pairs at $d = 3$, deep inside the capacity, where every non-bottleneck family is perfect. And, with the confound-free probe (Section 4.4), we can finally *isolate* the effect: holding the distance and the density fixed, fewer routes (a higher resistance) is harder — beyond the capacity a single path fails where two or three succeed. What we still cannot do is read the bottleneck off a *natural* graph as a number on the spectral gap: on natural graphs the gap is collinear with the diameter (Thread 1), and the two "gap sweeps" we tried do not actually isolate it (barbell_var varies the clique density and bridge length, its gap being tiny and U-shaped; expander_var varies the gap together with the density). So the bottleneck/resistance is a real, separable cause — demonstrable with a built probe — but inseparable from the diameter and the density on the graphs one meets in the wild. The honest summary of Threads 1–2: difficulty is the diameter (capacity) and the density of Thread 1, plus a genuine bottleneck/cut effect that a controlled probe isolates but a natural graph confounds with its diameter.

5 Thread 3: the read-out — can a different head push the wall out?

Threads 1–2 found the model's main limit is the 3^L distance wall, a property of the model that steering the data cannot move (Report III). Here we ask whether the *read-out* — how the model turns its per-node embeddings h_i into a prediction $\hat{R}_{ij}$ — can move it. The **standard (linear)** read-out maps the embeddings to $\hat{R}_{ij}$ through a symmetrised linear layer, which in practice reads connectivity off essentially a single node's embedding. We compare it to a **similarity** read-out that instead *compares* the two embeddings,

$$\hat{R}_{ij} = \text{scale} \cdot \cos(h_i, h_j) + \text{bias},$$

and, separately, to an auxiliary **Laplacian-smoothness loss**. Both are non-standard, so we read them as diagnostics ("what is the trunk missing"), not as a recommended architecture.

Every table keeps the read-outs (and where relevant the ER vs mixed training) separate, per seed, as in Threads 1–2. This thread also collects the two remaining error types — the *cut* and the *degree heuristic* — because both are about what the read-out and the data can or cannot repair.

5.1 The similarity read-out doubles the reach radius

Where it helps. Tables 19–20 give whole-graph exact-match per seed, linear vs. similarity, on the families where the two *differ* (on the rest — ER, 2cliques, 1chain, . . . — both read-outs are ≈ 1.00). The similarity read-out helps exactly the families that require propagating a connection over a *long distance*: at $n = 40$ it lifts path_union ($0.26 \to 0.71$), chain_plus ($0.06 \to 0.65$), 2chains ($0 \to$ up to 0.49) and 2cycle ($0 \to 1.00$). It does *nothing* for the barbell (a bottleneck) or for 1cycle — a clean check: a single C_{40} has pairs at distance 20, just past the doubled reach $2 \cdot 3^L = 18$, while 2cycle's two C_{20} top out at distance 10 and are solved.

seed	path_un.	chain_pl.	2chains	2cycle	1cycle	barbell
linear read-out						
1000	0.25	0.00	0.00	0.00	0.00	0.00
2000	0.25	0.02	0.00	0.00	0.00	0.00
3000	0.30	0.16	0.00	0.00	0.00	0.00
4000	0.26	0.05	0.00	0.00	0.00	0.00
similarity read-out						
1000	**0.76**	**0.73**	0.00	**1.00**	0.00	0.00
2000	**0.76**	**0.73**	0.00	**1.00**	0.00	0.00
3000	**0.69**	**0.64**	**0.49**	**0.99**	0.00	0.00
4000	**0.62**	**0.48**	**0.35**	**1.00**	0.00	0.00

Table 19: $n = 40$**, mixed training**: whole-graph exact-match per seed, linear vs. similarity read-out, on the families where the two differ. The similarity read-out helps the long-range families and not the bottleneck (barbell) or 1cycle (distance $20 > 2 \cdot 3^L = 18$).

seed	path_union	chain_plus	barbell
linear read-out			
1000	0.91	0.78	0.00
2000	0.93	0.79	0.00
3000	0.92	0.79	0.00
4000	0.92	0.77	0.00
5000	0.93	0.78	0.00
6000	0.92	0.79	0.00
7000	0.92	0.80	0.00
8000	0.92	0.78	0.00
similarity read-out			
1000	**1.00**	**1.00**	0.00
2000	**0.95**	**0.87**	0.00
3000	**1.00**	**1.00**	0.00
4000	0.80	0.06	0.00
5000	**1.00**	**1.00**	0.02
6000	**1.00**	**1.00**	0.36
7000	**1.00**	**1.00**	0.00
8000	**1.00**	**1.00**	0.00

Table 20: $n = 20$**, mixed training**: same, per seed. Only path_union and chain_plus differ at $n = 20$ (the cycles and 2chains are too short to exceed the capacity, so both read-outs solve them); the barbell is unsolved by both. Seed 4000 is an unlucky similarity run.

The mechanism, distance by distance. Table 21 and Figure 6 trace it on the held-out chain_plus: the linear read-out breaks right at the capacity $3^L = 9$ and sits at a ≈ 0.55 floor; the similarity read-out stays perfect to ≈ 18, then dips and recovers. The reading: the linear read-out infers $\hat{R}_{ij}$ from a single embedding h_i, which carries a $\sim 3^L$-hop "light cone" of propagation; the similarity read-out compares *two* embeddings, each with its own 3^L cone, so the pair is resolved once the two cones overlap — a meet-in-the-middle at $\sim 2 \cdot 3^L$.

read-out	seed	≤ 9	10	12	14	16	18	20	24	28	32	36
linear	1000	1.00	0.56	0.59	0.59	0.57	0.56	0.58	0.75	0.84	0.99	0.99
linear	2000	1.00	0.52	0.54	0.54	0.52	0.51	0.52	0.71	0.81	0.96	0.91
linear	3000	1.00	0.57	0.60	0.58	0.54	0.52	0.51	0.70	0.81	0.97	1.00
linear	4000	1.00	0.52	0.54	0.54	0.53	0.51	0.52	0.72	0.82	0.97	0.99
similarity	1000	1.00	1.00	1.00	1.00	1.00	1.00	0.86	0.88	1.00	1.00	1.00
similarity	2000	1.00	1.00	1.00	1.00	1.00	1.00	0.87	0.88	1.00	1.00	1.00
similarity	3000	1.00	1.00	1.00	0.98	0.90	0.86	0.85	0.88	0.98	1.00	1.00
similarity	4000	1.00	1.00	0.98	0.91	0.82	0.77	0.76	0.79	0.90	0.97	1.00

Table 21: Reach by within-component distance on held-out **chain_plus**, $n = 40$, mixed training, four seeds. Both curves are non-monotone (worst in the middle, recovering farthest), which is why we report the whole profile, not a single "reach radius".

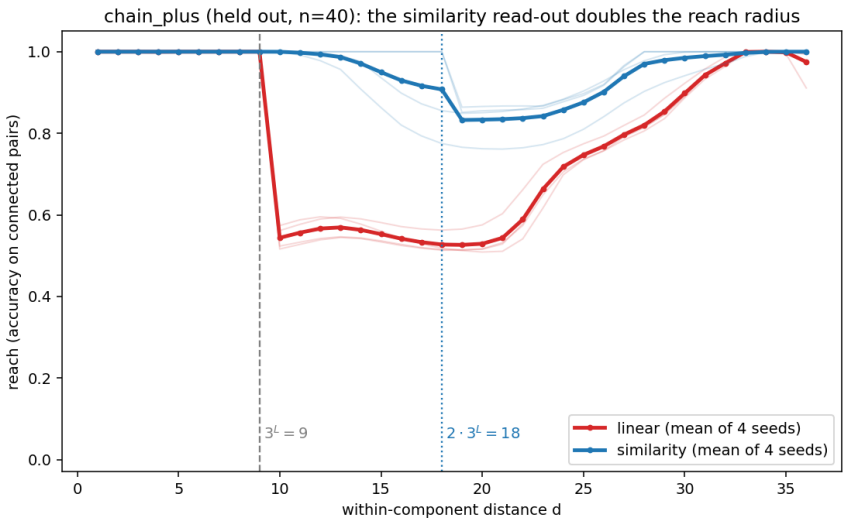

Figure 6: Table 21 drawn out (thick = mean of four seeds, thin = per seed). Linear (red) drops right after $3^L = 9$; similarity (blue) holds to $\approx 2 \cdot 3^L = 18$. Both recover on the farthest pairs.

Confirmed by depth. Repeating the Report III depth sweep ($n = 64$ path-union, vary the number of layers L) with the similarity read-out (Table 22, single seed) pins the factor: the linear read-out resolves to exactly 3^L ($d = 3$ at $L = 1$, $d = 9$ at $L = 2$), the similarity read-out to exactly twice that ($d = 6$, $d = 18$), then degrades. The wall still triples per layer, so the read-out buys a fixed factor of 2, not a change of regime. ($L = 3$ already saturates $n = 64$, so the two cannot be separated there.) The $d = 18$ point matches the $n = 40$ chain_plus bend, and in-distribution path-union exact rises $0.40 \to 0.62$ at $L = 2$.

L (3^L)	read-out	3	6	9	12	15	18	21	25	30	40	50	63
1 (3)	linear	1.00	0.79	0.79	0.80	0.81	0.82	0.83	0.84	0.86	0.91	0.93	1.00
1 (3)	similarity	1.00	**1.00**	0.69	0.70	0.71	0.72	0.73	0.75	0.77	0.82	0.89	1.00
2 (9)	linear	1.00	1.00	1.00	0.92	0.91	0.88	0.85	0.83	0.84	0.92	1.00	1.00
2 (9)	similarity	1.00	1.00	1.00	**1.00**	**1.00**	**1.00**	0.90	0.90	0.92	0.98	1.00	1.00

Table 22: Depth sweep, reach by distance on $n = 64$ path-union (seed 1000). The similarity read-out resolves to $2 \cdot 3^L$, double the linear 3^L, at both depths; bold marks the distances it resolves and linear does not.

What it cannot fix: the bottleneck. On the barbell the similarity read-out is no better than linear (exact 0 for both, every seed, Tables 19–20) and on the connected pairs it is even slightly *worse* (reach ≈ 0.30 vs ≈ 0.48): comparing two embeddings buys nothing when the information cannot cross the cut in the first place. That limit belongs to the trunk and would need more depth, not a different read-out.

5.2 The Laplacian-smoothness loss

A different lever is to add, to the cross-entropy, an auxiliary loss that directly pushes the embeddings of *adjacent* nodes together:

$$\mathcal{L}_{\text{smooth}} \;=\; \frac{1}{|E|} \sum_{(i,j)\in E} \|h_i - h_j\|^2 \;=\; \frac{1}{|E|}\,\mathrm{Tr}\big(H^\top L\, H\big),$$

where H stacks the node embeddings, $L = D - A$ is the graph Laplacian and $|E|$ the number of edges; the term is weighted by λ. The motivation is spectral: the kernel of L is exactly the per-component indicator vectors, so connected nodes "should" end up with equal embeddings.

The effect splits by training set (Table 23, λ swept over eight seeds). Under **mixed** training it is *inert* ($\lambda = 0$ vs 10^{-3}: $|\Delta| \leq 0.04$ per family), because the similarity read-out already drives the smoothness energy to ≈ 0 on its own. On the **ER** baseline it does act, as a *connection-vs-cut* lever: a small λ extends long-range reach (chain_plus 0.11 $\rightarrow$ 0.41, 1chain 0.64 $\rightarrow$ 0.92) but blurs the boundary the model needs to *separate* components, hurting the one family that needs a cut (2chains 0.47 $\rightarrow$ 0.20, its cut accuracy 0.99 $\rightarrow$ 0.94); a large $\lambda = 10^{-2}$ over-smooths and collapses everything. So it trades cut for connection rather than removing the wall, and is redundant once mixed training or the similarity read-out is used; we drop it for the main results.

ER-trained, similarity	$\lambda = 0$	$\lambda = 10^{-3}$
chain_plus (held out)	0.11	**0.41**
1chain	0.64	**0.92**
path_union	0.68	**0.82**
2chains	**0.47**	0.20

Table 23: The Laplacian-smoothness loss on the **ER** baseline (similarity read-out, $n = 20$, exact-match mean over 8 seeds). A small λ helps the long-range connection families and hurts the one needing a *cut* (2chains). Under mixed training the same comparison is flat ($|\Delta| \leq 0.04$).

5.3 Two ways to be wrong: under-reaching and over-connecting

The model can *under-reach* (miss a far-but-connected pair — the 3^L wall) or *over-connect* (wrongly join two components — a cut failure). These differ: a clean matrix-power model has the reach wall but cuts perfectly (separate components share no path at any length; we verified this with an oracle), so a cut failure means the embeddings leak. The standard model does both, on the same long families (Table 24): the linear read-out loses cut accuracy on chain_plus (0.89) and path_union (0.97). The similarity read-out improves reach *and* cut with no trade-off (chain_plus cut 0.89 $\rightarrow$ 0.99); the Laplacian loss does the opposite (it spends cut to buy reach). The read-out is the better lever. *What kind* of cut error this is — whether it depends on how close the two sides were — is tested in Section 5.4.

	reach (connected)		cut (disconnected)	
Family	linear	similarity	linear	similarity
2chains	0.71	**1.00**	1.00	1.00
chain_plus	0.80	**0.97**	0.89	**0.99**
path_union	0.89	**0.99**	0.97	**0.99**
2cycle	0.95	**1.00**	1.00	1.00
er_blocks	1.00	1.00	1.00	1.00

Table 24: Reach and cut accuracy, $n = 40$, mixed training, mean over four seeds. The similarity read-out improves both on the long families. By contrast the Laplacian loss buys reach by spending cut (ER 2chains: reach 1.00, cut 0.99 $\rightarrow$ 0.94, eight seeds).

5.4 A direct test of the cut: the near-miss probe

We split a single chain into two components by removing one edge, and measure cut accuracy on the cross-component pairs vs their *near-miss distance* (how far apart they would be if the edge

returned), with the reach curve on the same axis. A clean matrix-power model cuts perfectly at every near-miss distance (verified with an oracle), so a flat curve at 1.0 means clean matrix powering, while a dip at small near-miss distances would mean the embedding remembers the almost-path.

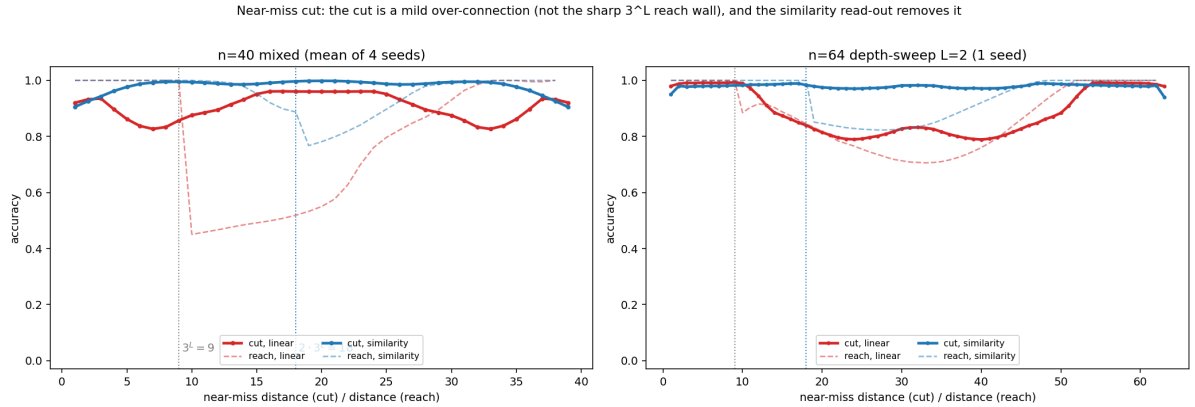

Figure 7: Near-miss cut test. Solid = cut accuracy vs near-miss distance; dashed = reach vs distance, same axis. Left: $n = 40$ mixed (mean of four seeds); right: $n = 64$ depth-sweep $L = 2$ (one seed). Reach drops sharply at capacity (9 linear, 18 similarity); the cut is a shallow, broad dip with no step there, and the similarity read-out flattens it to ≈ 1.0.

The cut is a mild, broad over-connection, not a second wall. (Figure 7.) The linear read-out does not cut perfectly — its cut accuracy dips to ≈ 0.8, so the embeddings do leak — but unlike the reach cliff at 9 the cut only sags gently and has no step at $3^L = 9$ or $2 \cdot 3^L = 18$; the dip is worst at *intermediate* distances (around 7 at $n = 40$, 25 at $n = 64$) and moves with the model, so it is a model-specific imperfection, not a function of the capacity. The similarity read-out flattens it to 0.95–1.0 everywhere. So reach and cut are genuinely different failures, and the similarity read-out cleans up both.

5.5 A third failure mode: the degree heuristic — fixed only by the data

Thread 1 found that graphs *denser* than the training set are out-of-distribution and fail (Section 3.2, Figure 3). The sharpest single instance of that is the **2cliques** family, and it belongs in this thread because it is the one failure the read-out and the loss *cannot* repair. On 2cliques the model has two jobs: *connect* the pairs inside each clique (distance 1, directly adjacent — the easiest case in the report) and *cut* the pairs across the two cliques. Table 25 splits the accuracy into those two jobs plus the whole-graph exact-match, for every training/read-out combination of this report ($n = 20$, eight seeds), including the Laplacian-loss run.

The standard **ER, linear** baseline fails even the easy within-clique $d = 1$ pairs (reach 0.71, and seed-dependent, 0.20–1.00): it mis-classifies *directly connected* nodes just because they sit in a dense clique where every node has a high degree — a degree-based shortcut, learned from sparse ER alone, that the high degree throws off. This is the *degree heuristic*, and it is the same density-out-of-distribution effect as in Thread 1, now pinned to one family. The interventions of this thread act on it as follows:

- the **similarity read-out** fixes the within-clique reach outright ($\rightarrow 1.00$) but then *over-connects* the two cliques (cut $\rightarrow 0$), so exact-match stays 0: never having seen a dense block it must split off, it does not know the cliques are separate;
- the **Laplacian loss** changes nothing here (identical to the read-out alone) — it helps long-range *connection*, which is not the problem; 2cliques needs a cut;

- only **mixed training** solves the whole family (reach, cut and exact all 1.00, either read-out), because the mixed set contains dense disconnected blocks (2cliques itself, clique_blocks) and so teaches both to connect inside a clique and to cut between cliques.

So this failure is repaired only by the right *data*, not by the read-out or the loss — the mirror image of the distance wall (which the read-out doubles) and unlike the bottleneck of Thread 2 (which nothing here touches).

training	read-out	within-clique reach (d=1)	across-clique cut	exact-match
ER	linear	0.71 (seed range 0.20–1.00)	0.89	0.38
ER	similarity	**1.00**	**0.00**	0.00
ER	similarity + loss (λ=10^{-3})	**1.00**	**0.00**	0.00
mixed	linear	**1.00**	**1.00**	**1.00**
mixed	similarity	**1.00**	**1.00**	**1.00**

Table 25: **2cliques**, $n = 20$, mean over the eight seeds. The ER/linear baseline fails the easy within-clique d=1 pairs (the degree heuristic). The similarity read-out — with or without the Laplacian loss — fixes that reach but then fails the cut (it over-connects the two cliques), so exact-match stays 0. Only mixed training solves the whole family, with either read-out: it is fixed by the data, not the head.

6 Takeaway

Difficulty (Threads 1–2). A graph is hard for the standard model along two axes we can separate: **distance vs. capacity** (the $3^L = 9$ wall — the model confirms a connection only within ~ 9 hops, sharply, as the per-seed tables show) and **density vs. training** (graphs denser than the ER training set are out-of-distribution and fail, the $n = 40$ model almost at once). The spectral gap is *not* a third axis one can read off a *natural* graph: there it is collinear with the diameter (-0.92), and the two naive "gap sweeps" (barbell_var, expander_var) turn out to vary the density and the diameter, not the gap. But a real bottleneck does add difficulty, and a confound-free probe — fixed distance, fixed density, only the number of parallel routes varied — *isolates* it: beyond the capacity a single route fails where two or three succeed. So the bottleneck/resistance is a genuine, separable cause when we construct for it, even though a natural graph confounds it with the diameter.

Architecture (Thread 3). The **similarity read-out** ($\hat{R}_{ij} \propto \cos(h_i, h_j)$) doubles the reach radius, from 3^L to $2 \cdot 3^L$ (measured at $L = 1, 2$ and on the held-out chain_plus at $n = 40$), because it compares two 3^L-hop neighbourhoods that meet in the middle; it also removes the model's over-connection error (the cut), with no trade-off — but it cannot cross a bottleneck (the barbell stays at 0). The **Laplacian loss** is redundant once the read-out or mixed training is used; on the plain ER baseline it only trades reach for cut. Both are non-standard, so we read them as diagnostics, not a recommended architecture. The **degree heuristic** (failing the easy within-clique pairs of dense graphs) is repaired at the pair level by either the read-out or mixed training, but the whole 2cliques family is solved only by mixed training — the right *data*, not a different head.